



TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the [document register](#) for the full explanation.

Clause: 5.1 — Software Development Planning · **Register:** [Document Register](#)

What the standard requires (Class C — every sub-clause of 5.1)

REF	TITLE	APPLIES AT	OUTPUT
5.1.1	Software development plan	A, B, C	Software development plan
5.1.2	Keep software development plan updated	A, B, C	Software development plan, kept updated as development proceeds
5.1.3	Reference to system design and development	A, B, C	System requirements referenced in the plan, and procedures for coordinating software development with system development
5.1.4	Standards, methods and tools planning	C only	Standards, methods and tools for Class C software items, included or referenced in the plan
5.1.5	Integration and integration testing planning	B, C	Plan to integrate the software items (including SOUP) and to test during integration

REF	TITLE	APPLIES AT	OUTPUT
5.1.6	Software verification planning	A, B, C	Verification planning: deliverables requiring verification, the verification tasks, milestones, acceptance criteria
5.1.7	Software risk management planning	A, B, C	Plan to conduct the software risk management process, including risks relating to SOUP
5.1.8	Documentation planning	A, B, C	For each document: title/naming convention, purpose, and procedures/responsibilities for development, review, approval, modification
5.1.9	Software configuration management planning	A, B, C	What is controlled, activities, responsible organisations, when items come under control, when problem resolution is used
5.1.10	Supporting items to be controlled	B, C	Tools, items and settings used to develop the software, included in the items to be controlled
5.1.11	Configuration item control before verification	B, C	Plan to place configuration items under CM control before they are verified
5.1.12	Identification and avoidance of common software defects	B, C	Procedure for identifying relevant defect categories, and evidence they do not contribute to unacceptable risk

This project's development plan

5.1.1–5.1.2 — the plan and keeping it current. This document set *is* the plan, split by process area rather than kept as one file, so each part can be updated independently as the corresponding area of the project changes — which is itself a documentation-planning decision, covered under 5.1.8 below.

5.1.3 — coordination with system-level development. There is no separate "system" above the software here — the training site is the whole product. The nearest equivalent is the relationship between course *content* (`data/*.json` , sourced from the standard itself) and the *software* that presents it (`/*.js` , `/*.html` , `style.css`): content changes are treated as requirements changes and flow through [03 — Requirements](#) before the software is touched.

5.1.4 — standards, methods and tools (Class C).

- **Language/runtime:** plain HTML5, CSS3, ES2017+ JavaScript. No framework, no build step, no bundler — a deliberate choice recorded in the project's design notes so that "the site itself still has none" (no dependency) stays true.
- **Test tooling:** [Playwright](#) (`>=1.40` , see `tests/requirements.txt`) driving real Chromium/Chrome; [axe-core](#) 4.10.2 for accessibility.
- **Coding conventions:** documented inline as they're established — see the "why" comments throughout `learn.js` , `quiz.js` , `contact.js` and the JavaScript Architecture section of the project's design notes.
- **Editor/agent tooling:** developed with the assistance of Claude Code; every substantive change is reviewed against the standard's normative text before being accepted, not merely against "does it run."

5.1.5 — integration and integration test planning. Addressed in [07 — Integration & Testing](#), which also records why this project's page-script architecture makes a *separate* integration test phase largely redundant with system testing.

5.1.6 — verification planning. `tests/test_site.py` 's own docstring states the plan directly: three layers (Data, Behaviour, Quality), roughly half the checks deliberately exercising failure paths rather than only the happy path. See [08 — System Testing](#) for the full breakdown by group.

5.1.7 — risk management planning. See [11 — Risk Management File](#). The plan, in short: every change to `applicability.json` is cross-checked line by line against the normative text before merge, because that file is where a wrong answer would cause the most harm to a reader.

5.1.8 — documentation planning. This document set's own convention:

DOCUMENT	TITLE/NAMING	OWNER OF UPDATES
docs/NN-*.md	Numbered by clause order, one file per process area	Whoever changes the corresponding area of the project
DESIGN.md	Page-by-page and architectural rationale	Whoever changes layout, UX, or JS structure
README.md	User-facing project summary, testing guide, screenshots	Whoever changes a feature, adds a test group, or changes the visuals
tests/anomaly_log.csv	Machine-maintained, not hand-edited	Generated by tests/test_site.py on every run

5.1.9–5.1.11 — configuration management planning. See [12 — Configuration Management Plan](#) for the full answer; in brief, git is the configuration management system, every third-party tool used to build or test the site is version-pinned (tests/requirements.txt , the axe-core CDN URL), and nothing is considered "controlled" until it passes tests/test_site.py .

5.1.12 — avoiding common defect categories (Amendment 1). The categories this project actively guards against, each with a named test group in tests/test_site.py as evidence they don't recur: unvalidated external data (data , learn – async loading and failure), silent async failures (every route(...).abort() / 404 test), and regressions in previously-fixed content defects — the applicability group's "site refuses to render on contradictory data" tests exist specifically because this category of defect happened once already (see [11](#)).

Cybersecurity planning (Amendment 1 addition)

The plan's security posture is small because the attack surface is small: no server-side code, no user accounts, no data at rest beyond a visitor's own localStorage

(theme choice, training level, banner dismissal — see [12](#)). The one place data leaves the browser is the contact form, and its handling is planned, implemented and *tested* as a discrete concern — see [03 §Security-related requirements](#) and the contact test group.

Gaps

None specific to this document — the gaps that matter (no separate unit testing, no separate integration phase, partial detailed design, no release versioning scheme) are recorded where they're relevant, in [05](#), [06](#), [07](#) and [09](#) respectively, per 5.1.8's own principle: state a thing once, where it belongs.